TP de Synthèse Git & GitHub

Objectifs :

- Configurer l'environnement et l'authentification sécurisée.
- Maîtriser le flux de travail local (états et zones).
- Gérer les versions (Rollback et Historique).
- Collaborer via des branches et résoudre des conflits.
- Synchroniser avec un dépôt distant GitHub.

Partie 1 : Configuration et Sécurité (Le Socle)

1. **Identité :** Configurez globalement votre nom d'utilisateur et votre email pour que chaque commit soit correctement attribué.
2. **Sécurité GitHub (SSH) :** Générez une paire de clés SSH (**Ed25519**), ajoutez la clé publique sur GitHub et testez la connexion avec ssh -T git@github.com.

Partie 2 : Initialisation et États des fichiers (Le flux local)

4. **Initialisation :** Créez un dossier TP_Recap, accédez-y et transformez-le en dépôt Git. Vérifiez la création du dossier caché .git.
5. **Gestion de l'index :**
    - Créez un fichier projet.txt.
    - Utilisez la commande pour voir que le fichier est en état **"untracked"**.
    - Ajoutez-le à la **zone d'index (staging area)** et vérifiez qu'il est "prêt à être validé".
    - Effectuez votre premier commit avec le message "Initialisation du projet".
6. **Ignorer des fichiers :** Créez un fichier .gitignore. Configurez-le pour ignorer tous les fichiers se terminant par .log. Testez-le en créant un fichier erreur.log et vérifiez qu'il n'apparaît pas dans le git status.

Partie 3 : Rollback et Historique (La machine à remonter le temps)

7. **Annulation locale :** Modifiez projet.txt mais ne validez pas. Utilisez la commande spécialisée pour annuler ces modifications et revenir à l'état du dernier commit.
8. **Reset Hard :** Créez un commit contenant un fichier **fichier_inutile.txt**. Utilisez git reset --hard pour supprimer ce commit et le fichier physiquement de votre répertoire de travail.
9. **Reset Soft :** Modifiez un fichier et commitez. Utilisez git reset --soft HEAD^ pour annuler le commit mais garder vos modifications dans la zone d'index (pour les corriger par exemple).
10. **Historique :** Visualisez l'historique complet de manière condensée et sous forme de graphe. Utilisez également le reflog pour voir toutes les actions passées (même les resets).

Partie 4 : Branches, Fusion et Conflits (Le travail d'équipe)

11. **Branches :** Créez une branche **feature-calcul** et basculez dessus. Créez un fichier **calcul.py** et commitez-le sur cette branche.
12. **Mise en réserve :** Modifiez calcul.py, puis utilisez git stash pour mettre vos changements de côté sans faire de commit. Vérifiez que votre répertoire est propre, puis récupérez vos changements.
13. **Conflit de fusion :**
    - Sur la branche main, modifiez la ligne 1 de projet.txt et commitez.
    - Basculez sur feature-calcul, modifiez la même ligne 1 de projet.txt avec un texte différent et commitez.
    - Tentez de fusionner feature-calcul dans main.
    - Identifiez les séparateurs de conflit (<<<<<<<, =======, >>>>>>>), résolvez manuellement le conflit dans le fichier, indexez et terminez la fusion par un commit.
14. **Squash Merge :** Créez une nouvelle branche, faites deux commits rapides, puis fusionnez-la dans main en utilisant l'option --squash pour ne créer qu'un seul commit propre dans l'historique final.

Partie 5 : Synchronisation GitHub (Le dépôt distant)

15. **Liaison :** Créez un dépôt vide sur GitHub nommé TP_Recap. Liez votre dépôt local à ce dépôt distant sous le nom **origin**.
16. **Push :** Envoyez votre branche main vers GitHub.
17. **Pull :** Sur l'interface GitHub, créez un fichier README.md directement en ligne. Revenez sur votre terminal et récupérez ce nouveau fichier en local.